Offline blend between run H0 and run H2

Combines the OOFs of the two hybrid runs (same folds, same rows) and measures with **nested** blending whether mixing the models of the two runs or averaging them brings a gain beyond noise.

Input (two folders, each with `oof_resnet.npy`, `oof_ft_transformer.npy`, `oof_catboost.npy`, `oof_targets.npy`, `cv_fold_of_row.npy`):

- `H0`: hybrid CV protocol, hyperparameters of the earlier hybrid run H0 (OOF nested blend 0.01720)
- `H2`: hybrid CV protocol, hyperparameters of run H2 (0.01715)

Decision rule: a variant is worth it only if it improves upon hybrid-hybrid baseline by at least `MIN_GAIN` on average **and** on at least `MIN_FOLDS_BETTER` out of 5 folds (paired differences per fold).

In [1]:
```python
# Step 1 - configuration
import os, json
import numpy as np
import pandas as pd
from scipy.optimize import minimize

H0_RUN = "/kaggle/input/datasets/marcomerlo37/moa-diagnostics-dataset"
H2_RUN = "/kaggle/input/datasets/marcomerlo37/moa-oof-dataset"
CLIP_EPS = 1e-15
MIN_GAIN = 5e-5
MIN_FOLDS_BETTER = 4
MODEL_FILES = {"resnet": "oof_resnet.npy", "ft": "oof_ft_transformer.npy", "cb": "oof_catboost.npy"}
```

In [2]:
```python
# Step 2 - load and verify that the two runs are aligned
def load_run(folder):
    return {"Y": np.load(f"{folder}/oof_targets.npy"),
            "fold": np.load(f"{folder}/cv_fold_of_row.npy"),
            **{name: np.load(f"{folder}/{file}") for name, file in MODEL_FILES.items()}}

h0_run, h2_run = load_run(H0_RUN), load_run(H2_RUN)
assert h0_run["Y"].shape == h2_run["Y"].shape and (h0_run["Y"] == h2_run["Y"]).all(), "different rows/targets between the two runs"
assert (h0_run["fold"] == h2_run["fold"]).all(), "different folds between the two runs: paired comparison is invalid"
Y, fold_of_row = h2_run["Y"], h2_run["fold"]
N_FOLDS = int(fold_of_row.max()) + 1
splits = [(np.where(fold_of_row != f)[0], np.where(fold_of_row == f)[0]) for f in range(N_FOLDS)]
print(f"Rows {Y.shape[0]} | targets {Y.shape[1]} | folds {N_FOLDS} | validation sizes {[len(v) for _, v in splits]}")
```

```
OOF = {f"{m}_{tag}": run[m] for tag, run in (("H0", h0_run), ("H2", h2_run)) for m in MODEL_FILES}
for pair in (("resnet_H0", "resnet_H2"), ("ft_H0", "ft_H2"), ("cb_H0", "cb_H2")):
    OOF[pair[0].split("_")[0] + "_avg"] = 0.5 * (OOF[pair[0]] + OOF[pair[1]])
```

```
Rows 21948 | targets 206 | folds 5 | validation sizes [4386, 4356, 4371, 4383, 4452]
```

In [3]:

```
# Step 3 - metric and nested blend (same functions as training notebook, but silent and with per-fold score)
def log_loss_mean(y, p, eps=CLIP_EPS):
    p = np.clip(np.asarray(p, dtype=np.float64), eps, 1 - eps)
    return float(np.mean(-(y * np.log(p) + (1 - y) * np.log(1 - p))))

def optimize_weights(oof_list, y):
    def objective(logits):
        w = np.exp(logits - logits.max()); w /= w.sum()
        return log_loss_mean(y, sum(wi * p for wi, p in zip(w, oof_list)))
    result = minimize(objective, x0=np.zeros(len(oof_list)), method="Nelder-Mead",
                      options={"xatol": 1e-6, "fatol": 1e-9, "maxiter": 3000})
    w = np.exp(result.x - result.x.max())
    return w / w.sum()

def nested_blend(names):
    """Weights optimized on training folds only, score on left-out fold. Returns per-fold scores and average weights."""
    oof_list = [OOF[n] for n in names]
    scores, weights = [], []
    for train_idx, val_idx in splits:
        w = optimize_weights([p[train_idx] for p in oof_list], Y[train_idx])
        scores.append(log_loss_mean(Y[val_idx], sum(wi * p[val_idx] for wi, p in zip(w, oof_list))))
        weights.append(w)
    sizes = [len(v) for _, v in splits]
    return np.array(scores), float(np.average(scores, weights=sizes)), np.mean(weights, axis=0)
```

In [4]:

```
# Step 4 - variants to compare
VARIANTS = {
    "H0 (all run H0)":                      ["resnet_H0", "ft_H0", "cb_H0"],
    "H2 (baseline, all run H2)":            ["resnet_H2", "ft_H2", "cb_H2"],
    "ResNet H2 + FT H0 + CatBoost H2":      ["resnet_H2", "ft_H0", "cb_H2"],
    "ResNet H2 + FT H0 + CatBoost H0":      ["resnet_H2", "ft_H0", "cb_H0"],
    "ResNet avg + FT avg + CB avg":         ["resnet_avg", "ft_avg", "cb_avg"],
    "6 models, free weights":               ["resnet_H0", "resnet_H2", "ft_H0",
                                             "ft_H2", "cb_H0", "cb_H2"],
}
results = {}
for label, names in VARIANTS.items():
    fold_scores, mean_score, mean_weights = nested_blend(names)
    results[label] = {"names": names, "fold_scores": fold_scores, "score": mean_score, "weights": mean_weights}
    print(f"{label:38s} nested {mean_score:.5f}")
```

```
# sanity check: baselines must reproduce numbers from training notebooks (H0 0.01714, H2 0.01715)
print("\nSanity check: expected H0 ~0.01714, expected H2 ~0.01715")
```

```
H0 (all run H0)                       nested 0.01720
H2 (baseline, all run H2)             nested 0.01715
ResNet H2 + FT H0 + CatBoost H2       nested 0.01715
ResNet H2 + FT H0 + CatBoost H0       nested 0.01715
ResNet avg + FT avg + CB avg          nested 0.01715
6 models, free weights                nested 0.01714

Sanity check: expected H0 ~0.01714, expected H2 ~0.01715
```

In [5]:

```
# Step 5 - paired comparison with baseline H2
base = results["H2 (baseline, all run H2)"]
rows = []
for label, res in results.items():
    diff = res["fold_scores"] - base["fold_scores"]          # negative = better than baseline
    rows.append({"variant": label, "nested": res["score"], "delta_vs_H2": res["score"] - base["score"],
                 "folds_better": int((diff < 0).sum()), "delta_per_fold(1e-5)": np.round(diff * 1e5, 0).astype(int).tolist(),
                 "adoptable": bool(label != "H2 (baseline, all run H2)" and -(res["score"] - base["score"]) >= MIN_GAIN
                                   and (diff < 0).sum() >= MIN_FOLDS_BETTER)})
report = pd.DataFrame(rows)
print(report.round(5).to_string(index=False))
print("\nAverage weights (order of models in variant):")
for label, res in results.items():
    print(f"  {label:38s} " + ", ".join(f"{n}={w:.3f}" for n, w in zip(res["names"], res["weights"])))
print(f"\nThreshold: gain >= {MIN_GAIN} and better on >= {MIN_FOLDS_BETTER}/{N_FOLDS} folds. "
      "Variants are chosen on the same OOF: gains close to the threshold should be treated as noise.")
```

```
                     variant  nested  delta_vs_H2  folds_better delta_per_fold(1e-5)  adoptable
               H0 (all run H0) 0.01720      0.00005             0     [5, 1, 3, 7, 10]      False
     H2 (baseline, all run H2) 0.01715      0.00000             0      [0, 0, 0, 0, 0]      False
ResNet H2 + FT H0 + CatBoost H2 0.01715     -0.00000             3    [-1, 0, -2, 0, 2]      False
ResNet H2 + FT H0 + CatBoost H0 0.01715      0.00000             3    [-1, 0, -2, 2, 3]      False
   ResNet avg + FT avg + CB avg 0.01715      0.00000             2     [0, -2, 0, 2, 2]      False
         6 models, free weights 0.01714     -0.00000             4   [-2, -1, -1, 0, 2]      False

Average weights (order of models in variant):
  H0 (all run H0)                       resnet_H0=0.298, ft_H0=0.485, cb_H0=0.217
  H2 (baseline, all run H2)             resnet_H2=0.478, ft_H2=0.347, cb_H2=0.175
  ResNet H2 + FT H0 + CatBoost H2       resnet_H2=0.477, ft_H0=0.354, cb_H2=0.169
  ResNet H2 + FT H0 + CatBoost H0       resnet_H2=0.459, ft_H0=0.367, cb_H0=0.174
  ResNet avg + FT avg + CB avg          resnet_avg=0.419, ft_avg=0.381, cb_avg=0.200
  6 models, free weights                resnet_H0=0.034, resnet_H2=0.416, ft_H0=0.208, ft_H2=0.149, cb_H0=0.089, cb_H2=0.106

Threshold: gain >= 5e-05 and better on >= 4/5 folds. Variants are chosen on the same OOF: gains close to the threshold should be treated as noise.
```